COMPTE RENDU DE PROJET

API Movie DB — CinéDB

Application web de consultation de films et d'acteurs

Étudiant	Mzé ABDALLAH SOILIHI
Formation	BTS SIO 1ère année — Option SLAM
Établissement	Lycée léonard de Vinci
Année scolaire	2025 — 2026
Date de rendu	10/04/2026

1. Introduction et contexte du projet

Dans le cadre de mon cours de développement web en 1ère année de BTS SIO option SLAM, j'ai réalisé un projet centré sur la consommation d'une API REST externe. Le sujet nous a été donné par notre professeur : créer une application web en PHP permettant de consulter des informations sur les films et les acteurs en interrogeant l'API The Movie Database (TMDB).

Ce projet m'a permis de travailler plusieurs compétences importantes du programme BTS SIO SLAM :

- La consommation d'une API REST avec récupération de données JSON
- Le développement d'interfaces web dynamiques en PHP
- La gestion des formulaires HTML (méthode GET, vérifications)
- La structuration d'un projet web en plusieurs fichiers PHP
- Le design d'interfaces utilisateur modernes avec CSS et Bootstrap

Le projet a été développé en local sur un serveur XAMPP (Apache + PHP), en utilisant l'API publique de The Movie Database disponible à l'adresse api.themoviedb.org.

2. Présentation de l'API TMDB

The Movie Database (TMDB) est une base de données collaborative de films et de séries télévisées. Elle propose une API REST gratuite (avec clé API) qui permet d'accéder à des millions de données : titres, affiches, bandes annonces, notes, acteurs, biographies, etc.

2.1 Fonctionnement général d'une API REST

Une API (Application Programming Interface) REST fonctionne sur le principe de requêtes HTTP. Le client (notre application PHP) envoie une requête vers une URL précise appelée endpoint, et le serveur répond avec des données structurées au format JSON.

Exemple d'endpoint utilisé pour récupérer les films populaires :

```
https://api.themoviedb.org/3/movie/popular?api_key=CLE_API&language=fr-FR
```

2.2 Le format JSON

Tous les résultats de l'API TMDB sont retournés au format JSON (JavaScript Object Notation). C'est un format texte structuré en paires clé/valeur très utilisé pour les échanges de données sur le Web. En PHP, on le convertit en tableau associatif grâce à la fonction json_decode().

Exemple de structure JSON retournée par l'API pour un film :

```
{
  "id": 939243,
  "title": "Sonic 3",
  "overview": "Description du film...",
  "poster_path": "/cheminImage.jpg",
  "vote_average": 7.8,
  "release_date": "2024-12-20"
```

```
}
```

2.3 Clé API et authentification

Pour utiliser l'API TMDB, il faut créer un compte sur themoviedb.org et générer une clé API personnelle. Cette clé est passée en paramètre dans chaque requête (paramètre api_key). Elle permet à TMDB d'identifier l'application qui fait la requête et de limiter les abus.

> Remarque : La clé API utilisée dans ce projet est une clé de test fournie par le professeur. En conditions réelles, cette clé ne doit jamais être publiée sur un dépôt Git public.

2.4 Endpoints utilisés dans le projet

Endpoint	Utilisation
/movie/popular	Films les plus populaires
/movie/top_rated	Films les mieux notés
/discover/movie?with_genres=ID	Films filtrés par genre
/movie/{id}	Détail complet d'un film
/movie/{id}/videos	Clé YouTube de la bande annonce
/movie/{id}/credits	Acteurs d'un film
/search/movie?query=MOT	Recherche de films par mot-clé
/search/person?query=MOT	Recherche d'acteurs par mot-clé
/person/{id}	Détail d'un acteur (bio, naissance...)
/person/{id}/combined_credits	Films joués par un acteur

3. Architecture et structure du projet

Le projet est organisé en plusieurs fichiers PHP, chacun ayant un rôle précis. J'ai choisi une architecture modulaire : les fonctions API sont centralisées dans un seul fichier, et chaque page inclut simplement l'en-tête et le pied de page communs.

3.1 Arborescence des fichiers

```
movieDB2026/
├── header.php          ← Navigation + CSS global
├── footer.php          ← Pied de page stylisé
├── get-proxy.php       ← Gestion requêtes HTTP (proxy lycée / maison)
├── fonctions.php       ← Toutes les fonctions d'appel à l'API
├── index.php         ← Page d'accueil (carrousel + grille par genre)
├── film.php            ← Détail d'un film
├── acteur.php          ← Fiche d'un acteur
├── topRated.php        ← Films les mieux notés
├── genreMovies.php     ← Films par genre
├── searchMovies.php    ← Résultats recherche films
└── searchActeurs.php   ← Résultats recherche acteurs
```

3.2 Rôle des fichiers principaux

Fichier	Rôle détaillé
header.php	Contient le DOCTYPE HTML, les imports CSS (Bootstrap, Google Fonts), tout le CSS global de l'application, et la barre de navigation fixe avec les menus et les formulaires de recherche.
footer.php	Ferme les balises HTML, affiche le pied de page avec les liens de navigation, les réseaux sociaux et le crédit TMDB.
get-proxy.php	Définit la fonction getProxy() qui tente d'abord une connexion directe (maison), puis utilise le proxy du lycée si la connexion directe échoue.
fonctions.php	Centralise toutes les fonctions qui appellent l'API TMDB : popularMovies(), topRatedMovies(), getMovieDetail(), getMovieTrailer(), getMovieCast(), searchMovies(), searchActeurs(), getActeurDetail(), getActeurFilms(), getActeurSuggestions().
popular.php	Page d'accueil. Affiche un carrousel automatique des 5 films les plus populaires, puis une grille des films classés par genre (films 6 à 25).
film.php	Page de détail d'un film. Reçoit l'identifiant du film via $_GET['id'], affiche la bande annonce intégrée, et les acteurs dans une barre scrollable.
acteur.php	Page de fiche acteur. Affiche la photo, la biographie, le top 4 des films de l'acteur et des acteurs suggérés issus de son film le plus populaire.

4. Gestion des requêtes HTTP — get-proxy.php

Un des premiers problèmes rencontrés lors du développement concernait les requêtes HTTP vers l'API TMDB. Au lycée, le réseau passe par un serveur proxy qui doit être déclaré explicitement pour que les requêtes HTTPS fonctionnent. À la maison, ce proxy n'existe pas et bloquerait les requêtes.

J'ai donc réécrit la fonction getProxy() pour qu'elle s'adapte automatiquement à l'environnement :

```
function getProxy($url) {
    // Tentative de connexion directe (fonctionne à la maison)
    $response = @file_get_contents($url);
    if ($response !== false) return $response;

    // Fallback : utilisation du proxy du lycée
    $options = ['http' => [
        'proxy' => 'tcp://172.16.0.54:8080',
        'request_fulluri' => true
    ]];
    $context = stream_context_create($options);
    $response = @file_get_contents($url, false, $context);
    return ($response !== false) ? $response : null;
}
```

Le caractère @ devant file_get_contents permet de supprimer les messages d'erreur PHP en cas d'échec, ce qui rend l'application plus propre visuellement. Si les deux tentatives échouent, la fonction retourne null, et chaque fonction appelante vérifie ce retour avant de continuer.

> Points importants : L'opérateur @ supprime les warnings PHP. La vérification 'if (!$r) return [];' dans chaque fonction évite les erreurs fatales si l'API est inaccessible.

5. Les fonctions API — fonctions.php

Le fichier fonctions.php centralise toutes les fonctions qui communiquent avec l'API TMDB. Chaque fonction suit la même logique : construction de l'URL, appel à getProxy(), décodage du JSON, et retour des données.

5.1 Logique commune à toutes les fonctions

```
function maFonction() {
    global $key;  // Clé API déclarée en haut du fichier
    $r = getProxy("https://api.themoviedb.org/3/...");
    if (!$r) return [];  // Sécurité si l'API ne répond pas
    return json_decode($r, true)['results'] ?? [];
}
```

Le paramètre true dans json_decode() est essentiel : il force le décodage en tableau associatif PHP plutôt qu'en objet stdClass. L'opérateur ?? [] est un opérateur de coalescence nulle : si la clé 'results' n'existe pas, on retourne un tableau vide plutôt qu'une erreur.

5.2 Détail des fonctions écrites

Fonction	Description et données retournées
popularMovies()	Retourne un tableau des 20 films les plus populaires. Chaque film : id, title, poster_path, backdrop_path, overview, vote_average, release_date, genre_ids.
topRatedMovies()	Retourne les 20 films les mieux notés. Même structure que popularMovies().
moviesByGenre($id)	Retourne les films d'un genre donné (identifiant numérique TMDB), triés par popularité décroissante.
getMovieDetail($id)	Retourne toutes les informations d'un film : titre, synopsis, genres (tableau d'objets), durée (runtime), note, date de sortie.
getMovieTrailer($id)	Retourne la clé YouTube (string) de la bande annonce. Cherche d'abord en français, puis en anglais. Retourne null si aucune vidéo trouvée.
getMovieCast($id)	Retourne les 20 premiers acteurs d'un film. Chaque acteur : id, name, character, profile_path.
searchMovies($query)	Recherche des films par mot-clé. Retourne un tableau de résultats avec les mêmes champs qu'un film populaire.
searchActeurs($query)	Recherche des personnes (acteurs, réalisateurs) par nom. Retourne : id, name, profile_path, known_for_department, known_for.
getActeurDetail($id)	Retourne la fiche complète d'un acteur : name, biography, birthday, place_of_birth, known_for_department, profile_path.
getActeurFilms($id)	Retourne les 4 films principaux d'un acteur, triés par popularité décroissante. Utilise l'endpoint combined_credits.
getActeurSuggestions($id)	Retourne 8 acteurs suggérés en récupérant le casting du film le plus populaire de l'acteur, puis en excluant l'acteur lui-même.
getPopularByGenre($movies)	Prend un tableau de films et les regroupe par genre. Retourne un tableau associatif [id_genre => ['name'=>..., 'movies'=>[...]]] pour afficher la grille par catégorie.

6. Page d'accueil — popular.php

La page d'accueil est la plus complexe du projet. Elle se compose de deux grandes parties : un carrousel automatique pour les 5 films les plus populaires, et une grille de films classés par genre pour les films 6 à 25.

6.1 Le carrousel (Top 5)

Le carrousel affiche les 5 films les plus populaires sous forme de diapositives plein écran. Il change automatiquement toutes les 30 secondes grâce à un timer JavaScript.

Chaque diapositive contient :

- L'image de fond du film (backdrop en résolution originale)
- Un dégradé sombre par-dessus pour rendre le texte lisible
- Le badge de classement (#1 Populaire, #2 Populaire...)
- Le titre du film en grand (police Bebas Neue)
- La note sur 10 et l'année de sortie
- Un court résumé du film (3 lignes maximum)
- Deux boutons : Bande annonce et Plus d'infos

Le bouton Bande annonce ouvre une modale (fenêtre superposée) contenant une iframe YouTube. La clé YouTube est récupérée au chargement de la page via la fonction getMovieTrailer() pour chacun des 5 films.

```
91
92  <!-- CAROUSEL -->
93  <div class="hero-carousel" id="heroCarousel">
94    <div class="carousel-slides" id="carouselTrack">
95      <?php foreach ($top5 as $i => $movie):
96        $bg = 'https://image.tmdb.org/t/p/original' . ($movie['backdrop_path'] ?? $movie['poster_path']);
97        $trailer = $trailers[$movie['id']] ?? null;
98        $year = substr($movie['release_date'] ?? '', 0, 4);
99        $note = round($movie['vote_average'] ?? 0, 1);
100       $desc = $movie['overview'] ?? '';
101     ?>
102     <div class="carousel-slide <?= $i===0?'active':'' ?>">
103       <div class="carousel-slide-bg" style="background-image:url('<?= $bg ?>')"></div>
104       <div class="slide-content">
105         <span class="slide-badge">#<?= $i+1 ?> Populaire</span>
106         <h1 class="slide-title"><?= htmlspecialchars($movie['title']) ?></h1>
107         <div class="slide-meta">
108           <span class="rating">★ <?= $note ?>/10</span>
109           <?php if ($year): ?><span><?= $year ?></span><?php endif; ?>
110         </div>
111         <p class="slide-desc"><?= htmlspecialchars($desc) ?></p>
112         <div class="slide-btns">
113           <?php if ($trailer): ?>
114             <button class="btn-gold" onclick="openTrailer('<?= $trailer ?>')">
115               <i class="bi bi-play-fill"></i> Bande annonce
116             </button>
117           <?php endif; ?>
118           <a href="film.php?id=<?= $movie['id'] ?>" class="btn-ghost">
119             <i class="bi bi-info-circle"></i> Plus d'infos
120           </a>
121         </div>
122       </div>
```

Activer Windows
Accédez aux paramètres pour activ

Code JavaScript du carrousel :

```
let current = 0;
const total = 5;

function goToSlide(n) {
  slides[current].classList.remove('active');
  current = (n + total) % total;  // Boucle infinie
  slides[current].classList.add('active');
  track.style.transform = `translateX(-${current * 100}%)`;
```

```
  clearInterval(timer);
  timer = setInterval(() => goToSlide(current + 1), 30000);
}
```

Le modulo (%) permet de faire boucler le carrousel indéfiniment : quand on dépasse le dernier slide, on revient automatiquement au premier.

6.2 La grille par genre (Top 6-25)

Les films du 6ème au 25ème sont affichés en dessous du carrousel, groupés par genre. La fonction getPopularByGenre() parcourt les films et les affecte au premier genre reconnu dans leur liste genre_ids.




```
<!-- TOP 6-25 PAR GENRE -->
<div style="padding: 3rem 2rem 0; max-width: 1400px; margin: 0 auto;">
  <?php if (empty($byGenre)): ?>
    <p style="color:var(--muted)">Impossible de charger les films.</p>
  <?php else: ?>
    <?php foreach ($byGenre as $gid => $group): ?>
    <div class="genre-section">
      <h2 class="section-title"><span class="acc"><?= $group['name'] ?></span></h2>
      <div class="movies-grid">
        <?php foreach ($group['movies'] as $movie):
          $poster = !empty($movie['poster_path'])
            ? 'https://image.tmdb.org/t/p/w342' . $movie['poster_path']
            : 'https://via.placeholder.com/342x513/13131a/7777aa?text=No+Image';
          $desc = mb_substr($movie['overview'] ?? '', 0, 100);
        ?>
        <div class="movie-card">
          <img src="<?= $poster ?>" alt="<?= htmlspecialchars($movie['title']) ?>" loading="lazy">
          <div class="movie-card-body">
            <div class="movie-card-title"><?= htmlspecialchars($movie['title']) ?></div>
            <div class="movie-card-desc">
              <?php if ($desc): ?>
                <?= htmlspecialchars($desc) ?>...
              <?php else: ?>
                Aucune description disponible.
              <?php endif; ?>
            </div>
            <a href="film.php?id=<?= $movie['id'] ?>" class="btn-voir">Voir plus</a>
          </div>
        </div>
        <?php endforeach; ?>
      </div>
    </div>
```

Chaque carte film affiche : l'affiche du film, le titre en gras, les 100 premiers caractères du résumé suivis de '...' et un bouton 'Voir plus' qui redirige vers film.php?id=XXX.

7. Page détail d'un film — film.php

Quand l'utilisateur clique sur 'Voir plus' ou 'Plus d'infos', il est redirigé vers film.php avec l'identifiant du film en paramètre GET : film.php?id=939243.

7.1 Vérification des données reçues

Avant d'appeler l'API, on vérifie que l'identifiant reçu est valide :

```
if (!isset($_GET['id']) || !ctype_digit($_GET['id'])) {
    header('Location: popular.php'); exit;
}
$id = (int) $_GET['id'];
```

La fonction ctype_digit() vérifie que la valeur est bien un nombre entier positif, ce qui protège contre les injections via l'URL. Le cast (int) assure une conversion propre.

7.2 Contenu de la page

La page se compose de trois sections :

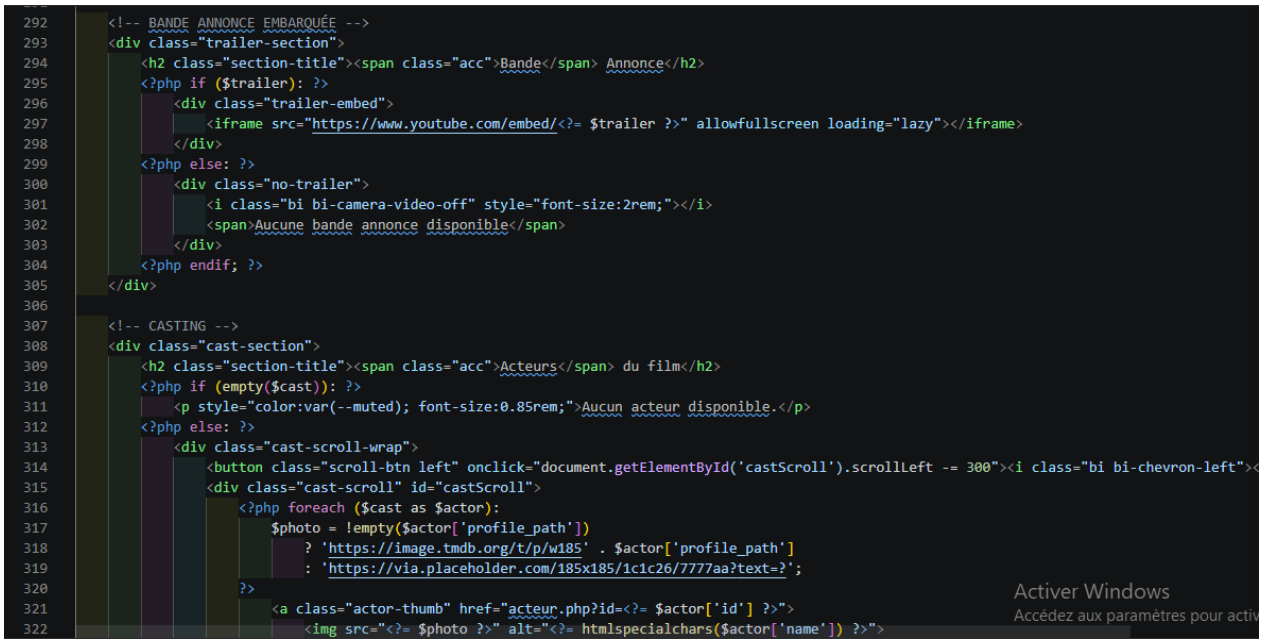
1. L'en-tête hero : grande image de fond (backdrop) du film avec un dégradé, sur laquelle sont affichés l'affiche du film, les genres, le titre, la note, la durée et le synopsis.
2. La bande annonce intégrée : une iframe YouTube est directement embarquée dans la page. Si aucune bande annonce n'est disponible, un message s'affiche à la place.
3. Le casting scrollable : les acteurs du film sont affichés dans une barre horizontale scrollable de droite à gauche. Chaque acteur est représenté par une photo circulaire, son nom et le personnage joué. Un clic redirige vers acteur.php?id=XXX.



```
<!-- HERO -->
<div class="film-hero"></div>

<div class="film-detail-wrap" style="padding-bottom:3rem;">
    <div class="film-detail-top">
        <div class="film-poster"><img src="<?= $poster ?>" alt="<?= $title ?>"></div>
        <div class="film-info">
            <?php if ($genres): ?>
                <div class="film-genres">
                    <?php foreach ($genres as $g): ?>
                        <span class="genre-tag"><?= htmlspecialchars($g) ?></span>
                    <?php endforeach; ?>
                </div>
            <?php endif; ?>
            <h1 class="film-title"><?= $title ?></h1>
            <div class="film-meta">
                <span class="rating">★ <?= $note ?>/10</span>
                <?php if ($year): ?><span><i class="bi bi-calendar3"></i> <?= $year ?></span><?php endif; ?>
                <?php if ($runtime > 0): ?><span><i class="bi bi-clock"></i> <?= $runtime ?> min</span><?php endif; ?>
            </div>
            <p class="film-overview"><?= $overview ?></p>
            <div class="film-btns">
                <?php if ($trailer): ?>
                    <button class="btn-gold" onclick="openTrailer('<?= $trailer ?>')">
                        <i class="bi bi-play-fill"></i> Bande annonce
                    </button>
                <?php endif; ?>
                <a href="index.php" class="btn-ghost"><i class="bi bi-arrow-left"></i> Retour</a>
            </div>
        </div>
    </div>
```

Activer Windows
Accédez aux paramètres pour activ



```
    <!-- BANDE ANNONCE EMBARQUÉE -->
    <div class="trailer-section">
        <h2 class="section-title"><span class="acc">Bande</span> Annonce</h2>
        <?php if ($trailer): ?>
            <div class="trailer-embed">
                <iframe src="https://www.youtube.com/embed/<?= $trailer ?>" allowfullscreen loading="lazy"></iframe>
            </div>
        <?php else: ?>
            <div class="no-trailer">
                <i class="bi bi-camera-video-off" style="font-size:2rem;"></i>
                <span>Aucune bande annonce disponible</span>
            </div>
        <?php endif; ?>
    </div>

    <!-- CASTING -->
    <div class="cast-section">
        <h2 class="section-title"><span class="acc">Acteurs</span> du film</h2>
        <?php if (empty($cast)): ?>
            <p style="color:var(--muted); font-size:0.85rem;">Aucun acteur disponible.</p>
        <?php else: ?>
            <div class="cast-scroll-wrap">
                <button class="scroll-btn left" onclick="document.getElementById('castScroll').scrollLeft -= 300"><i class="bi bi-chevron-left"><
                <div class="cast-scroll" id="castScroll">
                    <?php foreach ($cast as $actor):
                        $photo = !empty($actor['profile_path'])
                            ? 'https://image.tmdb.org/t/p/w185' . $actor['profile_path']
                            : 'https://via.placeholder.com/185x185/1c1c26/7777aa?text=?';
                    ?>
                        <a class="actor-thumb" href="acteur.php?id=<?= $actor['id'] ?>">
                            <img src="<?= $photo ?>" alt="<?= htmlspecialchars($actor['name']) ?>">
```

Activer Windows
Accédez aux paramètres pour activ

```
                        <div class="actor-thumb-name"><?= htmlspecialchars($actor['name']) ?></div>
                        <?php if (!empty($actor['character'])): ?>
                            <div class="actor-thumb-char"><?= htmlspecialchars(mb_substr($actor['character'], 0, 20)) ?></div>
                        <?php endif; ?>
                    </a>
                <?php endforeach; ?>
            </div>
            <button class="scroll-btn right" onclick="document.getElementById('castScroll').scrollLeft += 300"><i class="bi bi-chevron-right"
        </div>
    <?php endif; ?>
</div>
</div>
```

8. Page fiche acteur — acteur.php

La page acteur est ma fonctionnalité bonus. Elle nécessite deux appels API successifs : un pour les informations personnelles de l'acteur, et un autre pour récupérer sa filmographie.

8.1 Structure de la page

La page se décompose en quatre parties :

- Photo et informations : la photo de l'acteur s'affiche à gauche. À droite : son nom en grand, son département (Acting, Directing...), sa date et son lieu de naissance. En dessous, sa biographie complète avec scroll si le texte est long.
- Top 4 films : les 4 films les plus populaires joués par l'acteur, affichés sous forme de cartes horizontales (affiche à gauche, titre + résumé à droite). Un clic redirige vers la page du film.
- Acteurs suggérés : 8 acteurs issus du film le plus populaire de l'acteur. Affichés en grille de photos circulaires cliquables.
- Bouton retour : permet de revenir à la page précédente.

```
<!-- INFO ACTEUR -->
<div class="acteur-top">
    <div class="acteur-photo">
        <img src="<?= $photo ?>" alt="<?= $nom ?>">
    </div>
    <div>
        <h1 class="acteur-nom"><?= $nom ?></h1>
        <div class="acteur-meta">
            <?php if ($dept): ?><span><i class="bi bi-camera-reels"></i> <strong><?= $dept ?></strong></span><?php endif; ?>
            <?php if ($birth): ?><span><i class="bi bi-cake"></i> Né(e) le <strong><?= $birth ?></strong></span><?php endif; ?>
            <?php if ($lieu):  ?><span><i class="bi bi-geo-alt"></i> <strong><?= $lieu ?></strong></span><?php endif; ?>
        </div>
        <?php if ($bio && $bio !== 'Biographie non disponible.'): ?>
            <div class="acteur-bio"><?= $bio ?></div>
        <?php else: ?>
            <p style="color:var(--muted);font-size:0.85rem;">Biographie non disponible.</p>
        <?php endif; ?>
    </div>
</div>
```




```php
<!-- TOP 4 FILMS -->
<?php if (!empty($topFilms)): ?>
    <div style="margin-top:3.5rem;">
        <h2 class="section-title"><span class="acc">Top 4</span> Films</h2>
        <div class="top-films-grid">
            <?php foreach ($topFilms as $film):
                $fPoster = !empty($film['poster_path'])
                    ? 'https://image.tmdb.org/t/p/w185' . $film['poster_path']
                    : 'https://via.placeholder.com/90x130/1c1c26/7777aa?text=?';
                $fTitre  = htmlspecialchars($film['title'] ?? $film['name'] ?? 'Inconnu');
                $fDesc   = htmlspecialchars(mb_substr($film['overview'] ?? '', 0, 120));
                $fYear   = substr($film['release_date'] ?? $film['first_air_date'] ?? '', 0, 4);
                $fId     = $film['id'];
            ?>
                <a class="top-film-card" href="film.php?id=<?= $fId ?>">
                    <img src="<?= $fPoster ?>" alt="<?= $fTitre ?>">
                    <div class="top-film-info">
                        <div class="top-film-title"><?= $fTitre ?></div>
                        <?php if ($fDesc): ?>
                            <div class="top-film-desc"><?= $fDesc ?>...</div>
                        <?php endif; ?>
                        <?php if ($fYear): ?>
                            <div class="top-film-year"> <?= $fYear ?></div>
                        <?php endif; ?>
                    </div>
                </a>
            <?php endforeach; ?>
        </div>
    </div>
```

8.2 Gestion de la photo par défaut

Certains acteurs n'ont pas de photo dans la base TMDB. Dans ce cas, j'utilise une image placeholder générée dynamiquement :

```php
$photo = !empty($acteur['profile_path'])
    ? 'https://image.tmdb.org/t/p/w342' . $acteur['profile_path']
    : 'https://via.placeholder.com/342x513/1c1c26/7777aa?text=No+Photo';
```

9. Les formulaires de recherche

9.1 Rôle des attributs HTML du formulaire

Attribut	Rôle
action	Définit la page PHP qui va recevoir et traiter les données. Exemple : action="searchMovies.php" envoie le formulaire vers cette page.
method	Définit la méthode HTTP d'envoi. Ici on utilise GET : les données apparaissent dans l'URL (ex: searchMovies.php?query=sonic).
name	Identifie le champ du formulaire. La valeur saisie par l'utilisateur sera accessible en PHP via $_GET['query'] (si name="query").

9.2 Différence entre GET et POST

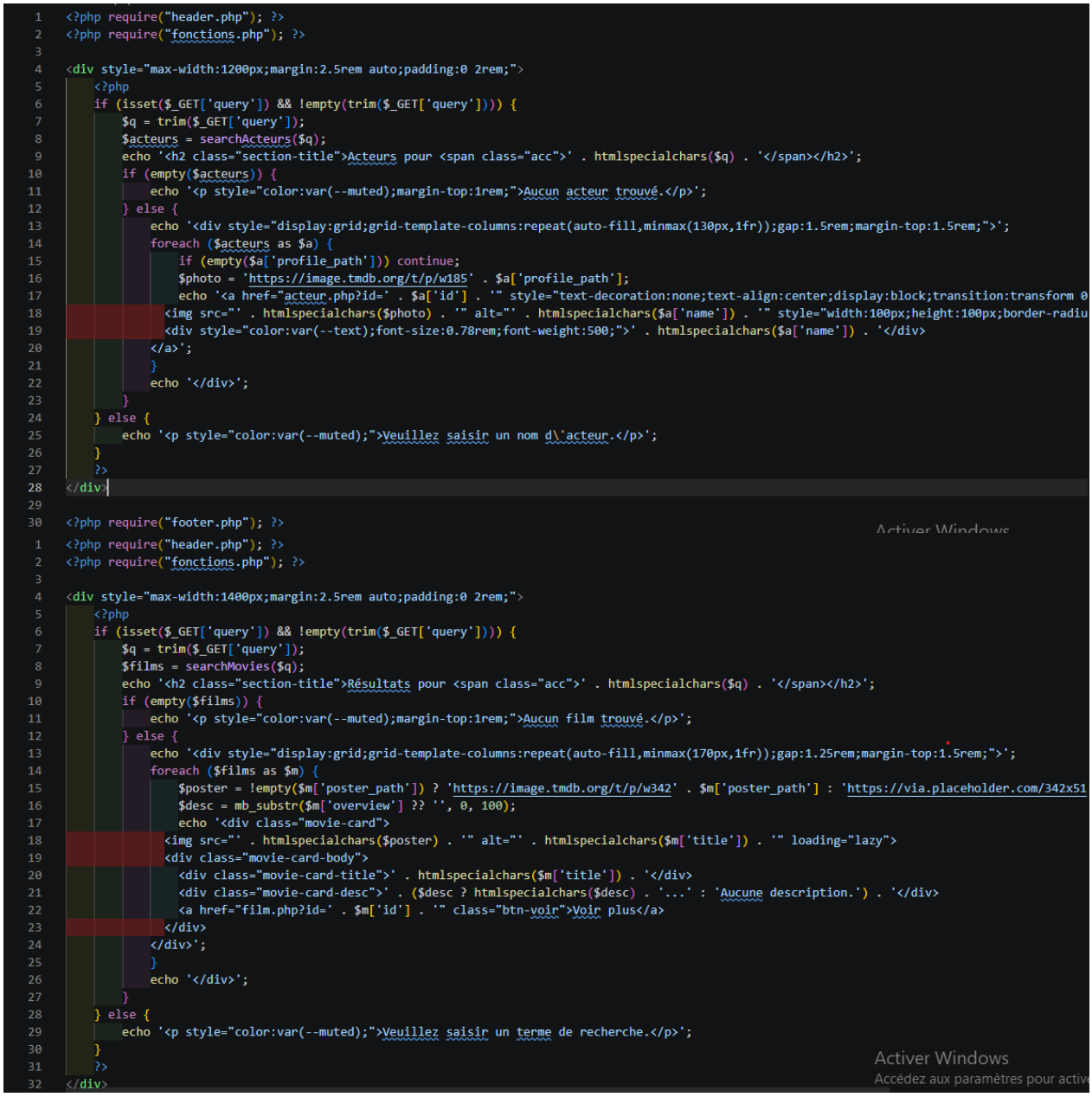
Critère	GET	POST
Données	Visibles dans l'URL (?query=tom)	Cachées dans le corps de la requête HTTP
Sécurité	Moins sécurisé (données dans l'URL, historique...)	Plus sécurisé pour les données sensibles
Usage	Recherches, filtres, partage d'URL	Formulaires de connexion, saisies sensibles
Notre projet	Utilisé pour les recherches (films, acteurs)	Non utilisé dans ce projet

9.3 Vérifications côté serveur

Sur les pages de résultats, je vérifie systématiquement les données reçues avant d'appeler l'API :

```
if (isset($_GET['query']) && !empty(trim($_GET['query']))) {
    $q = trim($_GET['query']);
    $films = searchMovies($q);
    // Affichage des résultats...
} else {
    echo "Veuillez saisir un terme de recherche.";
}
```

isset() vérifie que la variable existe. empty() vérifie qu'elle n'est pas vide. trim() supprime les espaces inutiles. htmlspecialchars() protège l'affichage contre les injections XSS.



```php
<?php require("header.php"); ?>
<?php require("fonctions.php"); ?>

<div style="max-width:1200px;margin:2.5rem auto;padding:0 2rem;">
    <?php
    if (isset($_GET['query']) && !empty(trim($_GET['query']))) {
        $q = trim($_GET['query']);
        $acteurs = searchActeurs($q);
        echo '<h2 class="section-title">Acteurs pour <span class="acc">' . htmlspecialchars($q) . '</span></h2>';
        if (empty($acteurs)) {
            echo '<p style="color:var(--muted);margin-top:1rem;">Aucun acteur trouvé.</p>';
        } else {
            echo '<div style="display:grid;grid-template-columns:repeat(auto-fill,minmax(130px,1fr));gap:1.5rem;margin-top:1.5rem;">';
            foreach ($acteurs as $a) {
                if (empty($a['profile_path'])) continue;
                $photo = 'https://image.tmdb.org/t/p/w185' . $a['profile_path'];
                echo '<a href="acteur.php?id=' . $a['id'] . '" style="text-decoration:none;text-align:center;display:block;transition:transform 0
                <img src="' . htmlspecialchars($photo) . '" alt="' . htmlspecialchars($a['name']) . '" style="width:100px;height:100px;border-radiu
                <div style="color:var(--text);font-size:0.78rem;font-weight:500;">' . htmlspecialchars($a['name']) . '</div>
            </a>';
            }
            echo '</div>';
        }
    } else {
        echo '<p style="color:var(--muted);">Veuillez saisir un nom d\'acteur.</p>';
    }
    ?>
</div>

<?php require("footer.php"); ?>
```

Activer Windows

```php
<?php require("header.php"); ?>
<?php require("fonctions.php"); ?>

<div style="max-width:1400px;margin:2.5rem auto;padding:0 2rem;">
    <?php
    if (isset($_GET['query']) && !empty(trim($_GET['query']))) {
        $q = trim($_GET['query']);
        $films = searchMovies($q);
        echo '<h2 class="section-title">Résultats pour <span class="acc">' . htmlspecialchars($q) . '</span></h2>';
        if (empty($films)) {
            echo '<p style="color:var(--muted);margin-top:1rem;">Aucun film trouvé.</p>';
        } else {
            echo '<div style="display:grid;grid-template-columns:repeat(auto-fill,minmax(170px,1fr));gap:1.25rem;margin-top:1.5rem;">';
            foreach ($films as $m) {
                $poster = !empty($m['poster_path']) ? 'https://image.tmdb.org/t/p/w342' . $m['poster_path'] : 'https://via.placeholder.com/342x51
                $desc = mb_substr($m['overview'] ?? '', 0, 100);
                echo '<div class="movie-card">
                <img src="' . htmlspecialchars($poster) . '" alt="' . htmlspecialchars($m['title']) . '" loading="lazy">
                <div class="movie-card-body">
                    <div class="movie-card-title">' . htmlspecialchars($m['title']) . '</div>
                    <div class="movie-card-desc">' . ($desc ? htmlspecialchars($desc) . '...' : 'Aucune description.') . '</div>
                    <a href="film.php?id=' . $m['id'] . '" class="btn-voir">Voir plus</a>
                </div>
            </div>';
            }
            echo '</div>';
        }
    } else {
        echo '<p style="color:var(--muted);">Veuillez saisir un terme de recherche.</p>';
    }
    ?>
</div>
```

Activer Windows
Accédez aux paramètres pour activ

10. Design de l'interface et navigation

10.1 Charte graphique

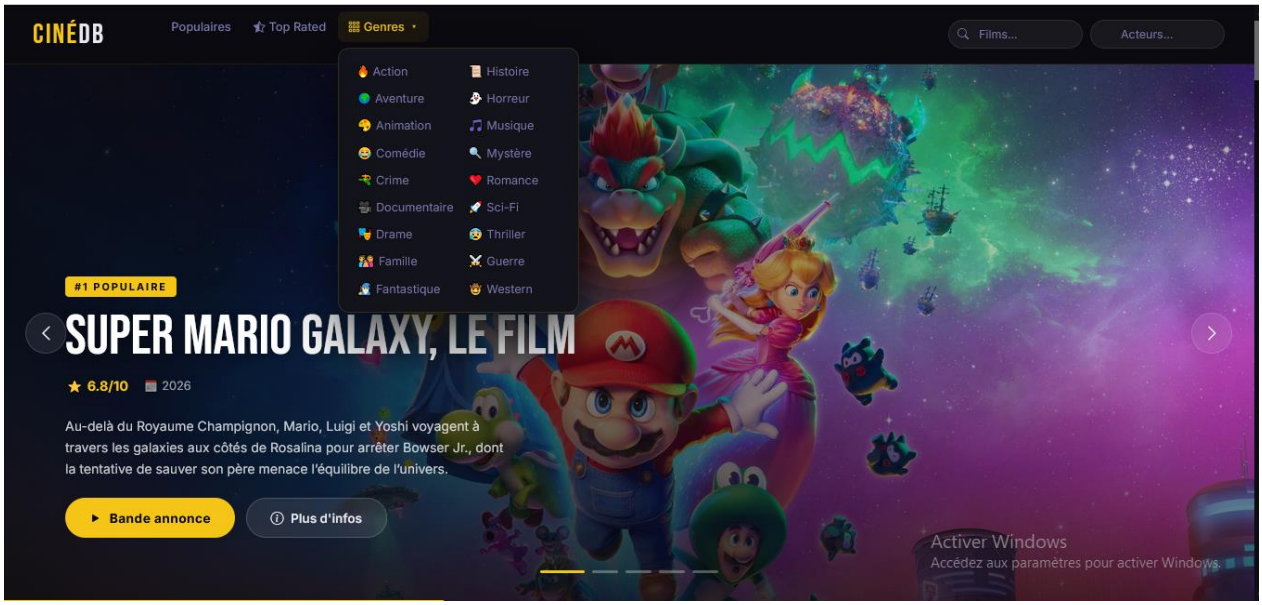
J'ai choisi un thème sombre inspiré des grandes plateformes de streaming (Netflix, Disney+), avec une palette de couleurs cohérente :

Élément	Couleur / Description
Fond principal	#0a0a0f — Noir profond
Fond cartes/nav	#13131a — Noir légèrement plus clair
Couleur accent	#f5c518 — Or (couleur IMDB)
Texte principal	#e8e8f0 — Blanc cassé
Texte secondaire	#7777aa — Gris-bleu

10.2 La barre de navigation

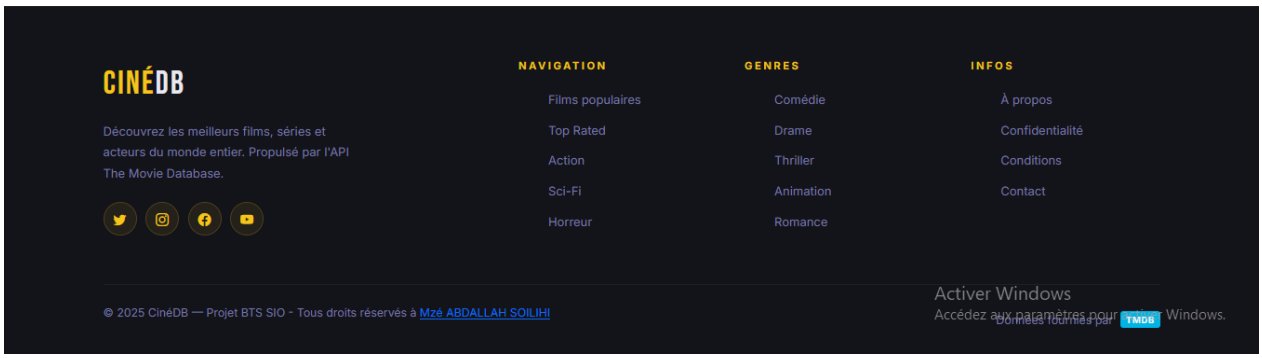
La navbar est fixe (position: fixed) et reste visible même en scrollant. Elle utilise un backdrop-filter: blur() pour un effet de verre dépoli moderne. Elle contient :

- Le logo CinéDB (police Bebas Neue, couleur or)
- Les liens de navigation : Populaires, Top Rated, et un menu déroulant Genres avec les 18 genres disponibles
- Deux barres de recherche : une pour les films, une pour les acteurs



10.3 Le footer

Le pied de page est organisé en 4 colonnes avec une grille CSS : le logo et description à gauche, puis trois colonnes de liens (Navigation, Genres, Infos). Le bas du footer affiche le copyright et le crédit TMDB.



11. Difficultés rencontrées et solutions apportées

Problème rencontré	Solution apportée
Les requêtes HTTPS échouaient au lycée à cause du proxy réseau.	Réécriture de getProxy() pour tenter d'abord une connexion directe, puis basculer automatiquement sur le proxy du lycée en cas d'échec.
Les warnings PHP s'affichaient dans la page quand l'API ne répondait pas, ce qui cassait la mise en page.	Ajout de l'opérateur @ devant file_get_contents() pour supprimer les warnings, et vérification du retour null dans chaque fonction.
Certains films du top 6-25 appartenaient aux mêmes genres, ce qui créait des sections vides ou déséquilibrées.	Limitation à 5 films maximum par genre dans getPopularByGenre(), et affichage uniquement si la section contient au moins un film.
La bande annonce n'était pas toujours disponible en français.	Implémentation d'un fallback dans getMovieTrailer() : cherche d'abord en fr-FR, puis en anglais, puis prend la première vidéo YouTube disponible.
Certains acteurs n'avaient pas de photo dans la base TMDB.	Utilisation d'une image placeholder générée dynamiquement par le service via.placeholder.com avec les couleurs du thème de l'application.
Les pages topRated.php et genreMovies.php n'existaient pas, causant des erreurs 404.	Création des fichiers manquants en réutilisant les mêmes fonctions déjà présentes dans fonctions.php.

12. Bilan et conclusion

Ce projet m'a permis de mettre en pratique de nombreuses notions vues en cours de 1ère année de BTS SIO SLAM. C'est la première fois que je travaillais avec une API REST externe dans un projet complet, et j'ai dû faire face à des problèmes concrets que l'on rencontre dans les projets professionnels : gestion du réseau, format de données, affichage conditionnel, et sécurité des entrées utilisateur.

En termes de compétences développées, je retiens particulièrement :

- La compréhension des échanges HTTP et du format JSON, qui sont des bases indispensables pour le développement web moderne
- L'organisation d'un projet PHP en fichiers modulaires et réutilisables
- La sécurisation des formulaires avec isset(), empty(), ctype_digit() et htmlspecialchars()
- La création d'interfaces utilisateur dynamiques et responsives sans framework lourd
- La gestion des cas d'erreur (API inaccessible, données manquantes, acteur sans photo...)

Si je devais améliorer le projet, j'ajouterais une pagination pour afficher plus de films, un système de favoris avec stockage en session PHP, et une page de détail pour les séries (l'API TMDB couvre aussi les séries TV).

> Ce projet a été réalisé entièrement en PHP/HTML/CSS/JS natif, sans framework back-end (pas de Laravel ni Symfony). Cela m'a permis de bien comprendre les mécanismes de base avant d'aborder des outils plus avancés.

13. Sources et ressources utilisées

Documentation officielle

- Documentation API TMDB — https://developer.themoviedb.org/docs
- Documentation PHP officielle (json_decode, file_get_contents, stream_context_create) — https://www.php.net/manual/fr/
- Documentation Bootstrap 5 (grilles, composants) — https://getbootstrap.com/docs/5.0/
- Bootstrap Icons — https://icons.getbootstrap.com/

Vidéos et tutoriels

- Grafikart — Tutoriel PHP pour débutants : https://www.youtube.com/watch?v=rFOCCM7sLyI
- Traversy Media — PHP REST API Introduction : https://www.youtube.com/watch?v=OEWXbpUMODk
- Net Ninja — CSS Flexbox & Grid Tutorial : https://www.youtube.com/c/TheNetNinja
- OpenClassrooms — Cours PHP/MySQL : https://openclassrooms.com/fr/courses/918836-concevez-votre-site-web-avec-php-et-mysql

Outils utilisés

- XAMPP (serveur Apache + PHP local) — https://www.apachefriends.org/fr/index.html
- Visual Studio Code (éditeur de code) — https://code.visualstudio.com/
- The Movie Database — Création du compte et clé API — https://www.themoviedb.org/
- Google Fonts — Police Bebas Neue et Inter — https://fonts.google.com/
- Placeholder.com — Images par défaut générées dynamiquement — https://via.placeholder.com/
- MDN Web Docs — Référence HTML/CSS/JS — https://developer.mozilla.org/fr/